

Series: STIG Hardening Docker Containers & Applications (Part 2 of 6)

Author: H. Brister Wooley

Published: 2026-04-01

Series: STIG Hardening Docker Containers & Applications (2/6)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-04-01-stig-hardening-docker-part2.md>

Last time we established the framework: three STIGs (Docker, Apache, PostgreSQL), the checklist-to-evidence mindset, and why Docker makes STIG weird.

Now we start where every container starts: the base image.

Your FROM line is the single most consequential STIG decision you'll make. It determines: - How many packages you inherit (each one a potential finding) - Whether FIPS mode works (V-235777, CAT I) - Whether you have sudo, su, cron out of the box (V-214230, CAT II) - How much time you spend justifying packages vs. building features

The Base Image Decision Matrix

Let's be honest about what you're actually getting:

Base Image	Size	Packages	FIPS	SUID/SGID Binaries	STIG Effort	Best For
ubuntu:24.04	~78 MB	~400	[OK] (kernel)	12+	High	Legacy apps needing glibc
alpine:3.20	~5 MB	~60	[WARN] (musl)	3-4	Medium	Most Go/Node/Python apps
registry.access.redhat.com/ubi9/ubi-minimal	~35 MB	~120	[OK] (RHEL)	5-6	Low	DoD/regulatory environments
gcr.io/distroless	~25 MB	~10	[OK]	0	Lowest	Productivity

Base Image	Size	Packages	FIPS	SUID/SGID Binaries	STIG Effort	Best For
distroless/cc-debian12	MB		(kernel)			on STIG workloads

My recommendation for STIG: Distroless. Here's why.

The Package Audit: What You're Actually Inheriting

Every package in your base image is a potential STIG finding. The auditor will run:

```
docker run --rm <your-image> rpm -qa      # RHEL/UBI
docker run --rm <your-image> apk info      # Alpine
docker run --rm <your-image> dpkg -l      # Debian/Ubuntu
```

And compare against: - **DISA STIG Package Lists** (unapproved packages = finding) -
SUID/SGID binaries (V-214230: "prevent execution of unauthorized software") -
Vulnerable versions (Grype/Trivy scan + CVE mapping)

Ubuntu 24.04: The "Why Are We Still Doing This" Baseline

```
$ docker run --rm ubuntu:24.04 dpkg -l | wc -l
~400 packages
```

SUID binaries out of the box:

```
$ docker run --rm ubuntu:24.04 find / -perm -4000 -type f 2>/dev/null
/usr/bin/sudo
/usr/bin/su
/usr/bin/mount
/usr/bin/umount
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

STIG impact: Each SUID binary is a potential privilege escalation vector (V-214230, CAT II). You must either remove them, justify them with a waiver, or implement compensating controls. That's *ten* findings before you've even written your first line of application code.

Alpine 3.20: Better, But Musl Breaks FIPS

```
$ docker run --rm alpine:3.20 apk info | wc -l
~60 packages
```

SUID binaries:

```
$ docker run --rm alpine:3.20 find / -perm -4000 -type f 2>/dev/null  
/bin/su  
/bin/busybox          # <- This one is sneaky
```

The busybox problem: Alpine's busybox is a single binary with SUID bit that provides su, mount, ping, traceroute, etc. One binary = multiple SUID paths. V-214230 finding.

The FIPS problem: Alpine uses musl libc, not glibc. OpenSSL FIPS module requires glibc. You *cannot* get FIPS 140-2 validated crypto on stock Alpine (V-235777, CAT I). Workarounds exist (install glibc, use WolfSSL), but they defeat the "small base image" purpose.

UBI 9 Minimal: The DoD Default

```
$ docker run --rm registry.access.redhat.com/ubi9/ubi-minimal rpm -qa | wc -l  
~120 packages
```

SUID binaries:

```
$ docker run --rm registry.access.redhat.com/ubi9/ubi-minimal find / -perm -  
4000 -type f 2>/dev/null  
/usr/bin/sudo  
/usr/bin/su  
/usr/bin/mount  
/usr/bin/umount  
/usr/bin/passwd
```

Advantages: - FIPS validated (RHEL kernel + OpenSSL FIPS module) – satisfies V-235777 - Red Hat maintains STIG alignment (UBI STIG content exists) - Long-term support (10 years)

Disadvantages: - Larger than Alpine - Still has sudo/su by default - Requires Red Hat subscription for updates (or EUS)

Distroless: The Endgame

```
$ docker run --rm gcr.io/distroless/cc-debian12 find / -perm -4000 -type f  
2>/dev/null  
# No output. Zero SUID binaries.
```

```
$ docker run --rm gcr.io/distroless/cc-debian12 ls /usr/bin  
# Only your app and glibc. No shell, no package manager, no `ls`, no `ps`.
```

What's inside: Your binary + glibc + CA certs + timezone data. That's it.

STIG wins: - V-214230 (privilege escalation): No SUID binaries = no finding - V-235777 (FIPS): Runs on FIPS-enabled host kernel, uses glibc + FIPS OpenSSL - V-214256 (file permissions): Read-only root FS enforced by design - V-214246 (least privilege): No shell = no interactive access = reduced attack surface

The catch: You can't docker exec into it. Debugging requires sidecars or ephemeral debug containers. Your CI/CD must be solid.

Hardening Any Base Image: The Universal Package Removal List

Regardless of base, these packages **must go** for STIG:

Remove privilege escalation vectors (V-214230, CAT II)

```
RUN apt-get purge -y sudo su passwd login cron || \  
apk del sudo su passwd login cron || \  
rpm -e --nodeps sudo passwd crontab
```

Remove network tools attackers love (V-214246, CAT II)

```
RUN apt-get purge -y netcat-openbsd nmap tcpdump wireshark || \  
apk del netcat-openbsd nmap tcpdump || \  
rpm -e --nodeps nmap-ncat tcpdump
```

Remove build tools (V-214230: unauthorized software)

```
RUN apt-get purge -y gcc make cmake build-essential || \  
apk del build-base || \  
rpm -e --nodeps gcc make cmake
```

Remove documentation (reduces image size, attack surface)

```
RUN rm -rf /usr/share/man /usr/share/doc /usr/share/info
```

Verification command for every build:

Must return empty

```
docker run --rm <your-image> find / -perm -4000 -type f 2>/dev/null
```

The FIPS Reality Check (V-235777, CAT I)

This is the one that kills Alpine-based STIG efforts:

V-235777: FIPS mode must be enabled on all Docker Engine - Enterprise nodes.

Check: `docker info | grep -i fips`

Expected: "Security Options: fips"

What this actually means for your container: 1. **Host OS must be FIPS-enabled** (RHEL 9 FIPS mode, Ubuntu 24.04 FIPS, or custom kernel) 2. **Your app must use FIPS-validated crypto** (OpenSSL FIPS module, BoringSSL FIPS, or WolfSSL FIPS) 3. **No non-FIPS algorithms anywhere** – not in TLS, not in hashing, not in signing

Distroless + FIPS OpenSSL (from gcr.io/distroless/fips-debian12)

```
FROM gcr.io/distroless/fips-debian12:latest
```

```
COPY --from=builder /app/myapp /myapp
```

```
USER nonroot:nonroot
```

```
ENTRYPOINT ["/myapp"]
```

```
# docker-compose.yml
services:
  app:
    image: myapp:fips
    security_opt:
      - "fips=on"           # Requires host FIPS mode
    cap_drop: ["ALL"]
    read_only: true
```

Verification:

```
# Inside container
openssl version -a | grep -i fips
# Should show: "FIPS provider version X.Y.Z"
```

What This Means for Your Strategy

The base image is your **first line of STIG defense**. Every package you inherit is a finding you must justify, mitigate, or remove.

Strategy: Start with distroless. If your app needs a shell for debugging, use a debug sidecar (gcr.io/distroless/cc-debian12-debug) in staging only. If you *must* use Alpine, document the FIPS gap and your compensating control (host FIPS + app-level crypto). Never use stock Ubuntu for STIG workloads – the package count alone will consume your audit budget.

What's Coming Next

Part 3: Dockerfile STIG controls. Every instruction (FROM, RUN, COPY, USER, HEALTHCHECK, EXPOSE) mapped to specific V-IDs from your project's Apache STIG and Docker STIG. The exact Dockerfile that passes all 47 Apache controls + Docker controls.

Plus: the one RUN command that eliminates 15 findings at once.

Read Part 3 ->

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Distroless base image selection	Domain 3: Security Architecture	3.2: Implement secure system design	Minimal attack surface, zero SUID binaries
Package audit with dpkg/apk/rpm	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Asset inventory and vulnerability identification
SUID/SGID binary removal	Domain 3: Security Architecture	3.2: Implement secure system design	Eliminates privilege escalation vectors (V-214230)
Network tool removal (nmap, tcpdump, nc)	Domain 3: Security Architecture	3.2: Implement secure system design	Reduces post-exploitation tooling
Build tool removal (gcc, make, cmake)	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents unauthorized software compilation
FIPS 140-2 validated crypto	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Meets V-235777 CAT I requirement
FIPS-enabled host + app-level crypto	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	End-to-end validated cryptographic module
Verification command in CI/CD	Domain 4: Security Operations	4.5: Implement automation/orchestration	Automated compliance gate on every build

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~60% | Domain 4 (Operations) – ~40%

Total Security+ Mapping: $\geq 50\%$ of content maps to SY0-701 objectives